

Business Entity Resolution

Determine which business records, arriving from three independent sources with no shared identifier, describe the same real-world business.

so we only rely on Business name & Address

THE PROBLEM

From sign-up to matched records

Amazon Business, create account

BUSINESS NAME

Acme Robotics Inc

BUSINESS ADDRESS

500 Market St, San Jose, CA

PHONE

EMAIL

Phone and email exist in reality. This challenge uses name and address only.

A business signs up. We capture its **name** and **address**.

THE PROBLEM

From sign-up to matched records

(clean)

SOURCE 1 reference, deduplicated	SOURCE 2 vendor A format	SOURCE 3 vendor B format
NAME Acme Robotics Inc.	NAME Acme Robotics Incorporated	NAME Acme Robotics
ADDRESS 500 Market St, San Jose	ADDRESS 500 Market Street, San Jose CA	ADDRESS Nr. City Hall, San Jose

noisy → must be reconciled against source 1

One business, three representations, each from a different data vendor, with **no shared ID**.

THE PROBLEM

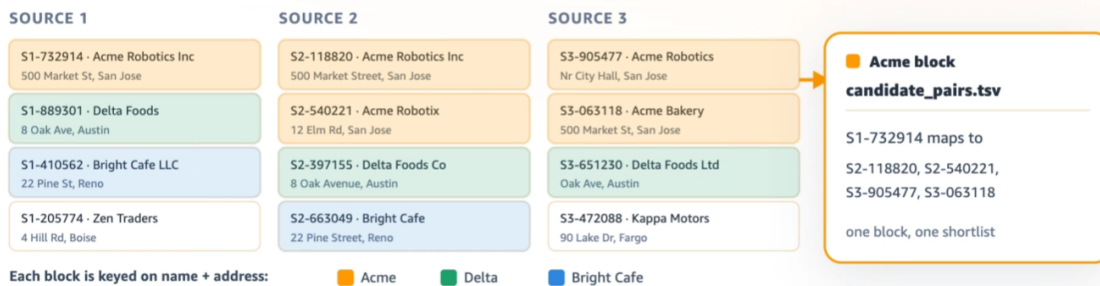
From sign-up to matched records



Entity resolution links every Source 1 entity to its matches: zero, one, or many.

THE PROBLEM

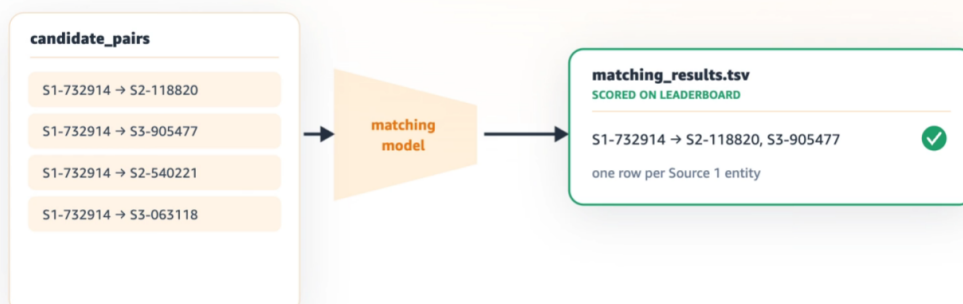
From sign-up to matched records



Blocking buckets records by a cheap key built from **name and address**, so similar names or shared addresses group together, matches and look-alikes alike.

THE PROBLEM

From sign-up to matched records



The **model** filters candidates and keeps only true matches, producing **matching_results.tsv**.

THE DATASET

Train to learn, **test to prove it**

● Training set

- Records across all **3 sources**
- **Ground-truth labels:** `train_ground_truth.tsv`
- Use it to build and evaluate your model

● Test set

- Same **3 sources, no labels**
- Produce a prediction for every Source 1 entity
- This is what the leaderboard scores

Ground truth is **one row per Source 1 entity**: its id maps to a comma-separated list of all matching Source 2 and Source 3 ids, so a single row holds the entity's full match set (empty when it matches nothing). All files are **tab-separated** (`.tsv`); read with `sep="\t"`.

● ● ● ● ●

3 / 6

WHAT YOU SUBMIT

A **file**, then a **package**

● During the challenge

- Upload `matching_results.tsv`
- **The only file scored** on the leaderboard
- One row per Source 1 entity, empty list means no match

● Final package (.zip)

- `output/matching_results.tsv`, final matches
- `output/candidate_pairs.tsv`, blocking set (audited)
- `code/...`, runnable pipeline
- `Documentation_template.md`, methodology

Run `utils/validate_submission.py` to catch any formatting errors before you submit.

● ● ● ● ●

4 / 6

JUDGING & TIPS

Scored on **macro $F_{0.5}$** , precision-weighted

$$F_{0.5} = (1.25 \times \text{Precision} \times \text{Recall}) / (0.25 \times \text{Precision} + \text{Recall})$$

- A **false merge** costs about twice as much as a miss, so when unsure, do not merge.
- **Singletons** are Source 1 entities with no match in Source 2 or 3. Predict an empty list for them and you earn a full 1.0; predict any match and you score 0.
- **Blocking sets your recall ceiling**, so invest there first. You cannot match a record you never consider.
- Account for **region-specific patterns** in both names and addresses.

⚠ **No external data lookup.** External databases, APIs, and geocoding services are strictly prohibited. Use only the provided data.

● ● ● ● ●

5 / 6